Name: Asha Raman Roll no: DA24B011

Q1. (c)

Q2. (b)

Q3. Output size = (W - K + 2P)/S + 1 = (64 - 5 + 4)/2 + 1 = 31.5 + 1, and taking the floor gives 32.
Output shape: 32 x 32 x 16.
Parameters: each filter has 5*5*3 = 75 weights + 1 bias = 76, so 16 * 76 = 1216.

Q4. Flattened size = 32 * 32 * 16 = 16384.
Weights = 16384 * 10 = 163840, biases = 10, total = 163850.

Q5. Let z = wx + b, so y = sigmoid(z).
dL/dw = dL/dy * dy/dz * dz/dw
dL/dy = (y - t), dy/dz = y(1 - y), dz/dw = x
So dL/dw = (y - t) y (1 - y) x.

Q6. During training, dropout sets each neuron's output to zero with probability p, so every mini-batch trains a different thinned network and neurons cannot rely on particular other neurons (no co-adaptation). At test time all neurons are kept and their outputs are multiplied by (1 - p) so the expected activation stays the same (inverted dropout divides by (1 - p) during training instead). Training many thinned networks that share weights is like averaging an ensemble, which reduces overfitting.

Q7.

```
import math
def softmax(x):
    m = max(x)
    exps = [math.exp(v - m) for v in x]
    s = sum(exps)
    return [e / s for e in exps]
```

Subtracting the max keeps every exponent <= 0, so math.exp never overflows for large inputs.